Name of the Student. Sumit..... Scholar Number. 241160111155

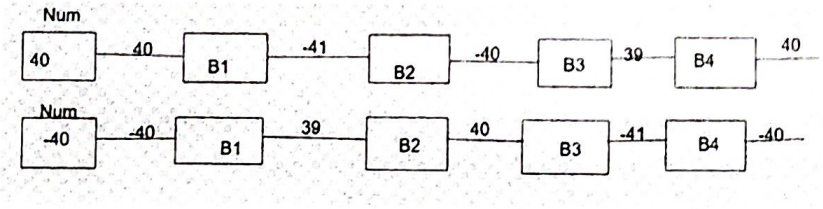
# MAULANA AZAD NATIONAL INSTITUTE OF TECHNOLOGY BHOPAL

## DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

**EXAMINATION: End Term Examination 2024-25** **MONTH and YEAR: May 2025**

**Course:** B. Tech **Semester:** II **Section:** F/G/H/I/J **Subject Code:** CSE-24105
**Subject Name: Problem Solving & Computer Programming** **Maximum Marks:** 40
**Duration:** 120 mins **Date: 15/05/25** **Time: 3-5PM**

**Note:** *Answer all questions. If required, take necessary assumptions and mention the same*

| Q. No. | Questions | Marks | COs |
|---|---|---|---|
| 1. | **(A)** Assume there are 100 Students in your class and the entire class voluntarily decided to donate blood at a blood donation camp organised at MANIT campus. There are some criteria for blood donation.<br>Criteria for both male student and female student which are common as follows.<br>a) The weight should be greater or equal to 45 Kg.<br>b) The age should be between 18 to 25 both inclusive.<br>c) Haemoglobin should be greater or equal to 12.5 g/dL.<br>d) Pulse should be between 50 to 100 per minute.<br>e) Body temperature should be less than 37.5 degree celsius.<br>In case, male student has already donated blood (blood_donated_status is yes) then the last_date_of_blood_donation should be more than 90 days to donate blood at MANIT campus. In case, female student has already donated blood (blood_donated_status is yes) then the last_date_of_blood_donation should be more than 120 days to donate blood at MANIT campus. Before blood donation, every student goes through a verification process to check whether the student is eligible to donate blood or not based on the above criteria.<br>Write an algorithm to check whether a student is eligible to donate blood. Your algorithm should also find out total male students and total female students who have donated blood and total number of non-eligible students who were unable to donate blood at MANIT campus. | 5 | CO1 |
| | **(B)**<br>  <br>See the above diagram and understand its execution. Block B1 takes input Num entered at run time and each block B2, B3, B4 takes input from its immediate left side block. All blocks execute a single C statement inside the block (different blocks may execute the same or different C statement but execute ONLY a single UNKNOWN statement. YOU NEED TO FIND & USE THE UNKNOWN STATEMENT of each block in your program) and produces output which acts as an input to the immediate next block. Block B4 produces output which is exactly the same as input Num.<br>Write a C program to take input Num from the user, execute the C statements inside the blocks, block by block as shown in the above diagram and display the output of block B1, B2, B3, B4.<br>NOTE: - (1) You can't use unary minus (-) and binary minus (-) operator in the whole program. | 5 | CO2 |

| | | | |
|---|---|---|---|
| 2. | **(A)** Write a program in C that takes 'n' integer numbers in the range of [1 … 1000] as input from the user. In the list of numbers entered by the user, some percentage 'p1' of numbers is divisible by 2 without any remainder, percentage p2 is divisible by 3 without remainder, percentage p3 is divisible by 4 without remainder. The program should compute and print the percentages of p1, p2 and p3.<br>*Input Data:* On the first line of input is the integer number n ($1 \leq n \leq 1000$) – count of numbers. On the next n lines, each line should have one integer number in the range of [1 … 1000]<br>*Output Data:* Print on the console 3 lines, each one containing a percentage between 0% and 100%, with accuracy up to two symbols after the decimal point<br><br>*Note:* In case of invalid number entered by the user, the program should print appropriate error messages. Arrays must not be used in this program. | **5** | **CO3** |
| | **(B)** Write a c program to sort a 2D array (4x4) row wise in ascending order.<br><br>Sample Input - / Sample Output -<br>8 5 7 2 / 2 5 7 8<br>7 3 0 1 / 0 1 3 7<br>8 5 3 2 / 2 3 5 8<br>9 4 2 1 / 1 2 4 9 | **5** | **CO3** |
| 3. | **(A)** Write a C program for Airline Seat Allocation System, where the airline has only a single row of 'n' seats represented by a boolean array. The index of the array represents the seat number, and the elements represent whether is the is free (0) or occupied (1). A passenger can request a seat by providing the seat number as input.<br>When a passenger requests a seat and the requested seat is already taken, the program should look for the next available seat in a circular fashion. This should be done recursively using pointers. If an available seat is allocated the program should print the allocated seat number otherwise if no seat is available, the program should print "No available seats". The program should take the following inputs: a) Total number of seats 'n' b) Availability of each seat (0 or 1) c) Requested Seat Number<br>For example:<br>Consider the following seat array:<br>Index / Seat Number → 0 1 2 3 4<br>Element / Availability → 1 0 0 1 1<br><br>Passenger requests for seat number 3 (represented by index 3). Since seat 3 is currently occupied, the next available seat is searched for. Seat 4 is also occupied, so searching again starts from seat 0 which is also full. Hence, the next available seat is Seat 1. | **5** | **CO5** |
| | **(B)** Write a C program to create a structure complex having real part and imaginary part of float type. The program should take two complex numbers as inputs from user, add them and display the result. | **5** | **CO6** |

CO3

| | | | |
|---|---|---|---|
| 4. | What will be the output of the following programs? In case of error, write down the reason for the error. | 5 x 2 | CO3, CO2, CO3, CO4 |

a)

```
#include <stdio.h>
void main()
{
   int i = 4;
   switch(i)
   {
      default:
         printf("This is default case\n");
      case 2:
         printf("This is second case\n");
         break;
      case 1:
         printf("This is first case\n");
          break;
      case 3:
          printf("This is third case\n");
          }
}
```

b)

```
#include <stdio.h>
int main()
{
int i= -3, j = 2, k = 0, m;
m = ++j && ++i || ++k;
printf ("\n%d %d %d %d", i, j, k, m )
;
return 0;
}
```

c)

```
#include <stdio.h>
int fun(int n)
{
   int i, j, sum = 0;
   for(i = 1;i<=n;i++)
      for(j=i;j<=i;j++)
         sum=sum+j;
   return(sum);
}

int main()
{
   printf("%d", fun(15));
   return 0;
}
```

d)

```
Assume that the array begins at memory location 65486.
#include <stdio.h>
int main()
{
   int arr[] = {12, 14, 15, 23, 45};
    printf ("%u %u", arr + 1, &arr + 1);
}
```

e)

```
#include <stdio.h>
   struct temp
   {
      int a;
   } s;
   void func(struct temp s)
   {
      s.a = 10;
      printf("%d\t", s.a);
   }
   int main()
   {
      func(s);
      printf("%d\t", s.a);
      return 0;
   }
```

**********END**********